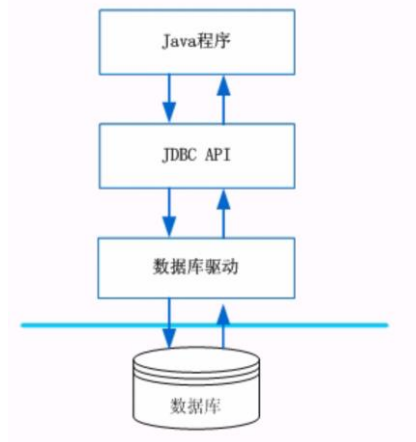


第六章 数据库 JDBC

JDBC (Java Data Base Connectivity 的缩写) 是 Java 程序操作数据库的 API, 也是 Java 程序与数据库交互的一门技术。JDBC 是 Java 操作数据库的规范, 由一组用 Java 语言编写的类和接口组成, 它对数据库的操作提供基本方法, 但对于数据库的细节操作由数据库厂商进行实现, 使用 JDBC 操作数据库, 需要数据库厂商提供数据库的驱动程序。



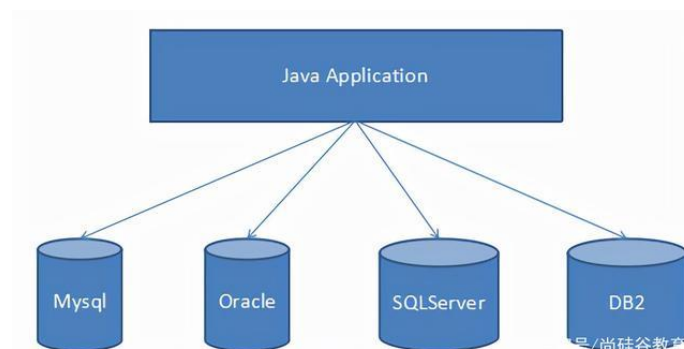
6.1 JDBC 概述

JDBC(Java Database Connectivity)是一个独立于特定数据库管理系统 (DBMS)、通用的 SQL 数据库存取和操作的公共接口 (一组 API), 定义了用来访问数据库的标准 Java 类库, 使用这个类库可以以一种标准的方法、方便地访问数据库资源。

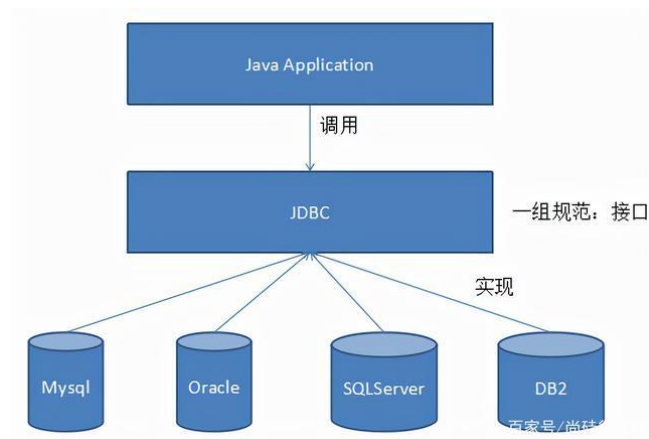
JDBC 为访问不同的数据库提供了一种**统一的途径**, 为开发者屏蔽了一些细节问题。

JDBC 的目标是使 Java 程序员使用 JDBC 可以连接任何**提供了 JDBC 驱动程序**的数据库系统, 这样就使得程序员无需对特定的数据库系统的特点有过多的了解, 从而大大简化和加快了开发过程。

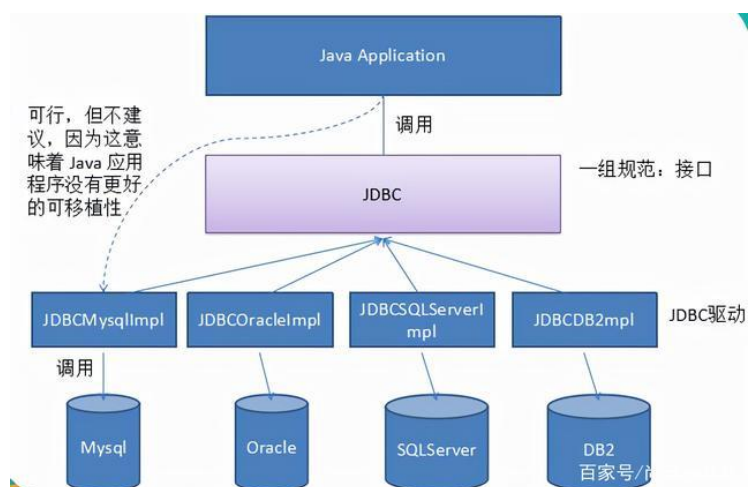
如果没有 JDBC, 那么 Java 程序访问数据库时是这样的:



改装:



实际上:

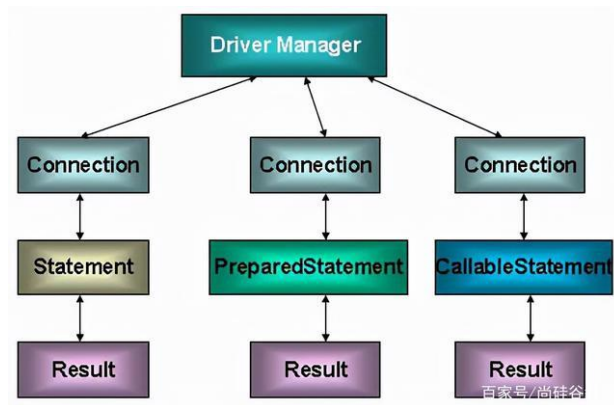


结论:

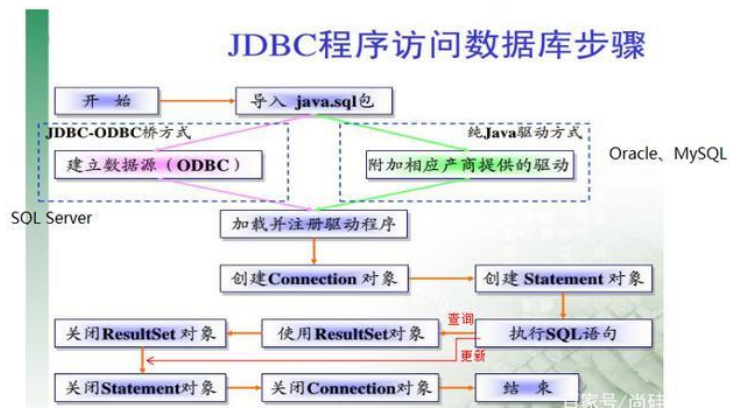
JDBC 是 SUN 公司提供一套用于数据库操作的接口 API, Java 程序员只需要面向这套接口编程即可。不同的数据库厂商, 需要针对这套接口, 提供不同实现。不同的实现的集合, 即为不同数据库的驱动。

6.2 JDBC 相关 API

JDBC API 是一系列的接口, 它统一和规范了应用程序与数据库的连接、执行 SQL 语句, 并得到返回结果等各类操作。声明在 java.sql 与 javax.sql 包中。



JDBC程序编写过程



6.3 Navicat 的使用

视频教程 1:

https://www.bilibili.com/video/BV1SL411Q7iU/?spm_id_from=333.788.recommend_more_video.3&vd_source=5ef6b6c8783e6e52622ec08ca403f89b

6.3 在 Eclipse 中配置 Mysql 驱动

1、首先去官网下载驱动：

<https://dev.mysql.com/downloads/connector/j/>

CSDN 上提供的下载网址：

<https://blog.csdn.net/kt1776133839/article/details/124539192>

下载 JDBC 的驱动 jar 文件，本例子采用的是：mysql-connector-java-8.0.13.zip，解压以后内容如下：

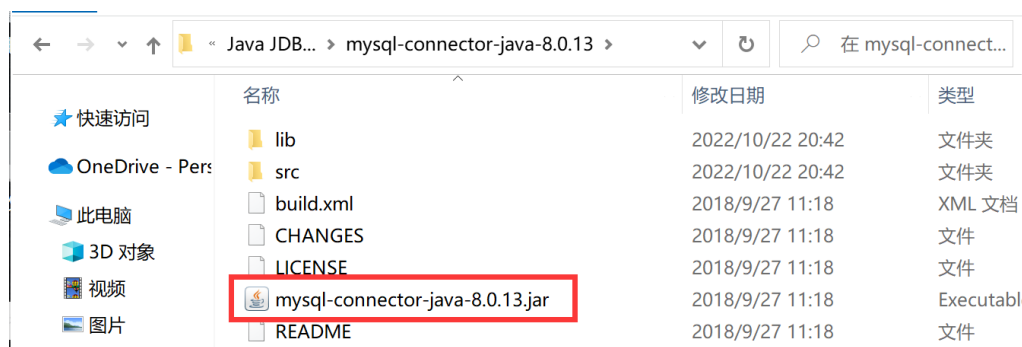


图 12-1 JDBC 驱动

2、创建一个 JDBC 工程项目

在菜单处选择 File->New->Project->Java Project，输入工程名称为：JdbcDemo，点击完成。

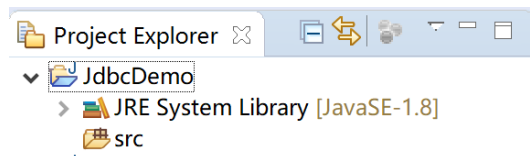


图 12-2 创建一个空的工程

创建好工程之后，点工程名 JdbcDemo 后右键，创建一个文件夹为 lib，用于存放图 12-1 的

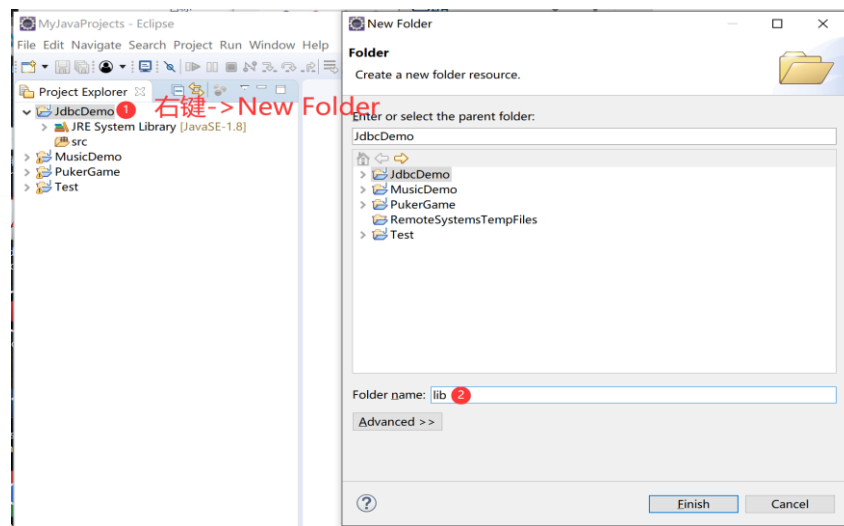


图 12-3 创建一个与 src 同级的目录 lib

jar 包，将图 12-1 中的 mysql-connector-java-8.0.13.jar 文件拷贝到工程目录下的 lib 文件夹中，然后回到 eclipse 界面，右击 lib 文件夹选择 Refresh 更新，这时候.jar 包文件便导入到工程中了，如下图所示：

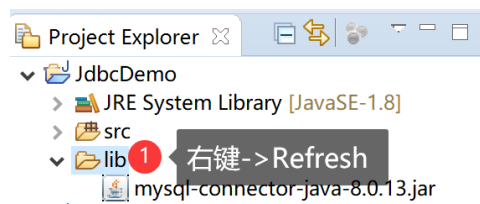


图 12-4 在 lib 文件夹中右键刷新

3、加载 jar 包

右击工程名 JdbcDemo 找到 Build path->configure build path, 点击 Libraries 之后将出现如下界面：点击 Add JARS, 将 mysql-connector-java-8.0.13.jar 添加即可，点击确定并应用，此时所有工作就完成了。

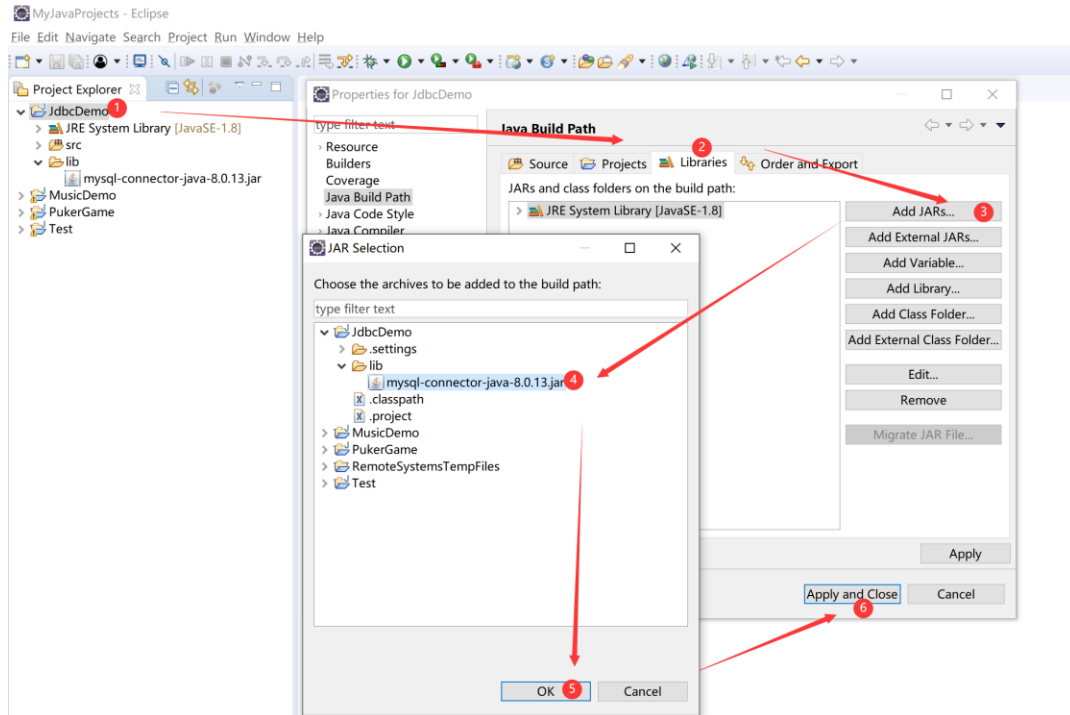


图 12-5 加载 jar 包

到此环境配置结束，可以在 eclipse 上新建一个工程，随便导入 `import java.sql.*`包，如果能导入成功，证明你的 MyAQL 包配置成功了。

6.4 MySQL 的相关指令

(1) 操作数据库

1. 查询 <pre>SHOW DATABASES;</pre>	3. 删除 - 删除数据库 <pre>DROP DATABASE 数据库名称;</pre> - 删除数据库(判断, 如果存在则删除) <pre>DROP DATABASE IF EXISTS 数据库名称;</pre>
2. 创建 - 创建数据库 <pre>CREATE DATABASE 数据库名称;</pre> - 创建数据库(判断, 如果不存在则创建) <pre>CREATE DATABASE IF NOT EXISTS 数据库名称;</pre>	4. 使用数据库 - 查看当前使用的数据库 <pre>SELECT DATABASE();</pre> - 使用数据库 <pre>USE 数据库名称;</pre>

use 数据库，进入数据库里面，使用数据库。

(2) 操作表

数据类型

分类	数据类型	大小	描述
数值类型	TINYINT	1 byte	小整数值
	SMALLINT	2 bytes	大整数值
	MEDIUMINT	3 bytes	大整数值
	INT或INTEGER	4 bytes	大整数值
	BIGINT	8 bytes	极大整数值
	FLOAT	4 bytes	单精度浮点数值
	DOUBLE	8 bytes	双精度浮点数值
	DECIMAL		小数
日期和时间类型	DATE	3	日期值
	TIME	3	时间值或持续时间
	YEAR	1	年份值
	DATETIME	8	混合日期和时间值
	TIMESTAMP	4	混合日期和时间值，时间戳
字符串类型	CHAR	0-255 bytes	定长字符串
	VARCHAR	0-65535 bytes	变长字符串
	TINYBLOB	0-255 bytes	不超过 255 个字符的二进制字符串
	TINYTEXT	0-255 bytes	短文本字符串
	BLOB	0-65 535 bytes	二进制形式的长文本数据
	TEXT	0-65 535 bytes	长文本数据
	MEDIUMBLOB	0-16 777 215 bytes	二进制形式的中等长度文本数据
	MEDIUMTEXT	0-16 777 215 bytes	中等长度文本数据
	LONGBLOB	0-4 294 967 295 bytes	二进制形式的极大文本数据
	LONGTEXT	0-4 294 967 295 bytes	极大文本数据

创建 create

创建表

```
CREATE TABLE 表名 (  
    字段名1 数据类型1,  
    字段名2 数据类型2,  
    ...  
    字段名n 数据类型n  
);
```

tb_user

id	username	password

注意：最后一行末尾，不能加逗号

查询 retrieve

查询表

- 查询当前数据库下所有表名称

```
SHOW TABLES;
```

- 查询表结构

```
DESC 表名称;
```

修改 update

修改表

1. 修改表名

```
ALTER TABLE 表名 RENAME TO 新的表名;
```

2. 添加一列

```
ALTER TABLE 表名 ADD 列名 数据类型;
```

3. 修改数据类型

```
ALTER TABLE 表名 MODIFY 列名 新数据类型;
```

4. 修改列名和数据类型

```
ALTER TABLE 表名 CHANGE 列名 新列名 新数据类型;
```

5. 删除列

```
ALTER TABLE 表名 DROP 列名;
```

tb_user

id	username	password

删除 delete

删除表

1. 删除表

```
DROP TABLE 表名;
```

2. 删除表时判断表是否存在

```
DROP TABLE IF EXISTS 表名;
```

操作数据， 添加、修改、删除

添加数据

1. 给指定列添加数据

```
INSERT INTO 表名 (列名1, 列名2, ...) VALUES (值1, 值2, ...);
```

2. 给全部列添加数据

```
INSERT INTO 表名 VALUES (值1, 值2, ...);
```

3. 批量添加数据

```
INSERT INTO 表名 (列名1, 列名2, ...) VALUES (值1, 值2, ...), (值1, 值2, ...), (值1, 值2, ...)...
```

```
INSERT INTO 表名 VALUES (值1, 值2, ...), (值1, 值2, ...), (值1, 值2, ...)...
```

修改数据

1. 修改表数据

```
UPDATE 表名 SET 列名1=值1, 列名2=值2, ... [WHERE 条件] ;
```

注意：修改语句中如果不加条件，则将所有数据都修改！

删除数据

1. 删除数据

```
DELETE FROM 表名 [WHERE 条件] ;
```

注意：删除语句中如果不加条件，则将所有数据都删除！

图形用户界面

Navicat

- Navicat for MySQL 是管理和开发 MySQL 或 MariaDB 的理想解决方案。
- 这套全面的前端工具为数据库管理、开发和维护提供了一款直观而强大的图形界面。
- 官网：<http://www.navicat.com.cn>

6.5 JDBC+MySQL 实现数据库管理

MySQL 是目前最火爆最受欢迎的开源数据库，是一个关系型数据库管理系统，由瑞典 MySQL AB 公司开发，属于 Oracle 旗下产品。SQL 结构化的查询语言（Structured Query Language），由 IBM 研发。MySQL, Oracle, DB2, SQL Server 等数据库都遵循 SQL 标准和规范，这就为数据库开发提供了方便。

DML 用于查询与修改数据记录，包括如下 SQL 语句：

- (1) INSERT: 添加数据到数据库中；
- (2) UPDATE: 修改数据库中的数据
- (3) DELETE: 删除数据库中的数据
- (4) SELECT: 查询数据，是 SQL 语言的基础，最为重要。

DCL 用来控制数据库的访问，包括如下 SQL 语句：

- (1) GRANT: 授予访问权限
- (2) REVOKE: 撤销访问权限
- (3) COMMIT: 提交事务处理
- (4) ROLLBACK: 事务处理回退
- (5) SAVEPOINT: 设置保存点
- (6) LOCK: 对数据库的特定部分进行锁定。

MySQL 软件的下载官网：

解压包 ZIP 格式下载网址：

<https://downloads.mysql.com/archives/community/>

安装包 msi 格式下载网址：

<https://downloads.mysql.com/archives/installer/>

MySQL 的数据库安装和卸载过程，请自行百度。

在成功安装好 MySQL 软件环境，配置好环境变量之后，接下来用 MySQL 创建一张表格。在命令行窗口 cmd 中，输入如下代码创建一张表格：

- (1) mysql -uroot -p123456 通过账户和密码进入 mysql 数据库指令
- (2) show databases; 显示当前的数据库，注意每条语句后面要输入分号
- (3) create database test; 创建一个叫 test 的数据库
- (4) show databases; 再次显示，发现多了一行叫 test 的数据库
- (5) use test; 操作创建的数据库，回车后发现指令改变了
- (6) show tables; 显示 test 数据库下的所有表格，

(7) `create table student(id int(10),name varchar(20));` 创建一个表格名为 student, 含有 id 和 name 两个字段, 并且规定了 id 是 int 类型, 长度为 10 个字符, name 的长度为 20 个字符

(8) `insert into student values(1001,'Jack');` 往表格中插入一行数据, 学号和姓名

(9) `insert into student values(1002,'Tom');` 往表格中插入一行数据, 学号和姓名

(10) `insert into student values(1003,'Smith');` 往表格中插入一行数据, 学号和姓名

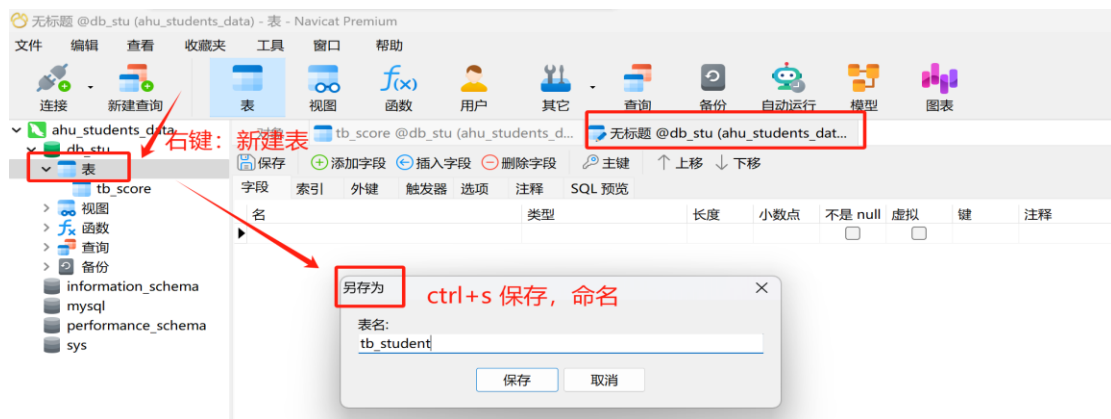
(11) `select * from student;` 查看表格所有的字段

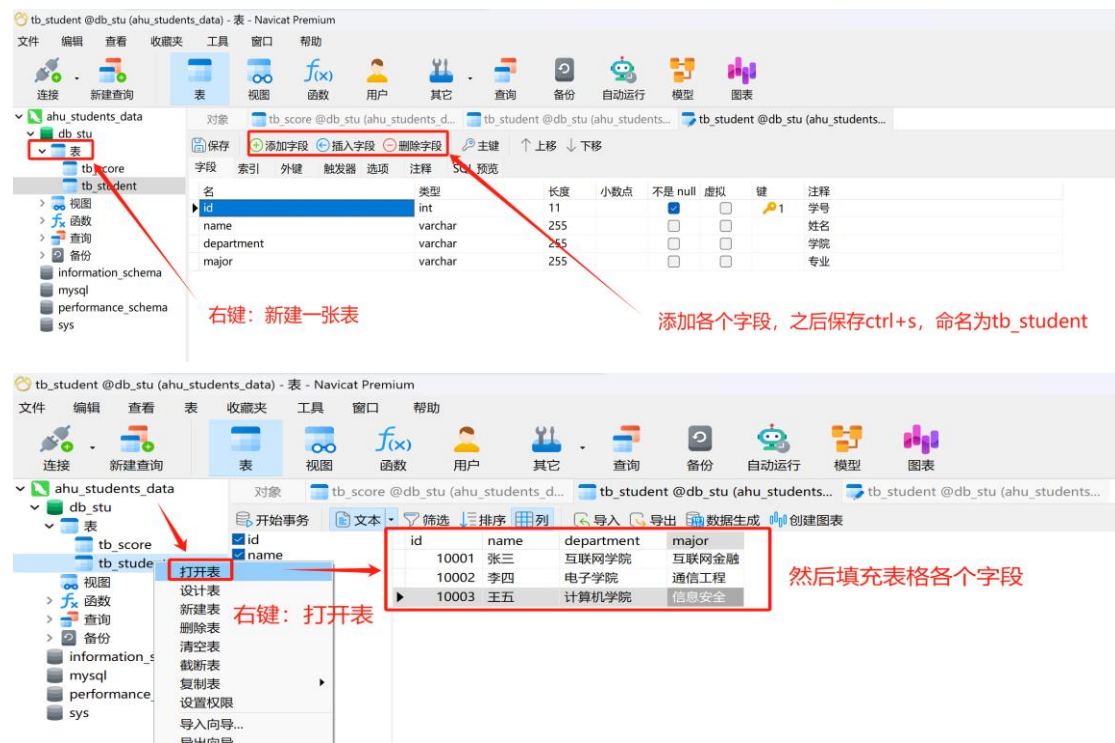
在了解 JDBC 与数据库后, 下面向大家介绍使用 JDBC 操作数据库的开发流程, 关键步骤如下:

- (1) 注册数据库驱动
- (2) 构建数据库连接 URL
- (3) 获取 Connection 对象

举例: 新建一张学生表, 并在 Java Project 控制台中调用, 实现控制台中的增删改查。

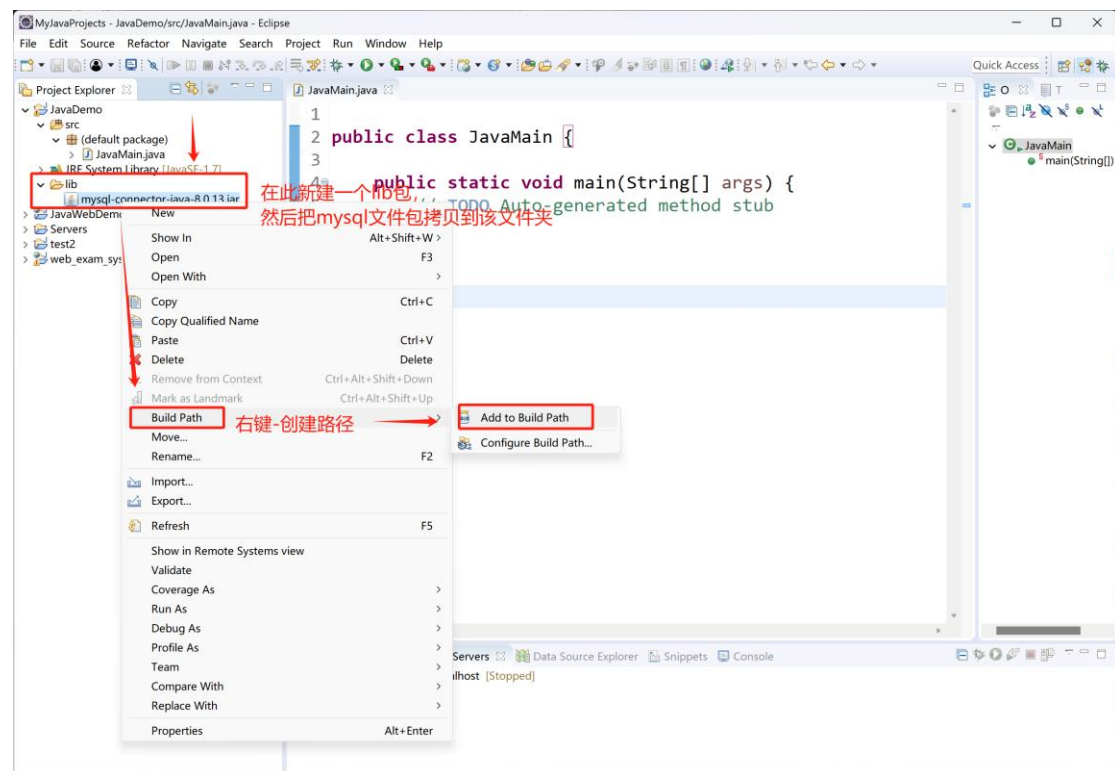
步骤 1: 用 navicat 新建一张学生表, 命名为 tb_student





注意：图中右键菜单选项，设计表为填写字段，打开表为填充表内容。经过上面的步骤，一张学生的表格就设计完毕了。

步骤 2：新建一个 Java Project 项目，在 src 平级的包中新建一个包，命名为 lib，并在 lib 中导入 mysql 的 jar 包。然后右键-Build Path-Add to Build Path，实现 jar 包的导入。



步骤 3：新建一个 JavaMain 类，开始写代码如下：

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
import java.util.Properties;
public class JavaMain {
    public static void main(String[] args) {
        try {
            connectDB();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

// 数据库的连接和使用
public static void connectDB() throws Exception{
    // 1. 获取数据库
    String driverClassName="com.mysql.cj.jdbc.Driver";
    Class cla=Class.forName(driverClassName);// 获取类型

    // 2. 连接数据库
    //String
    url="jdbc:mysql://localhost:3306/tb_student?useUnicode=true&characterEn
coding=utf8&serverTimezone=GMT%2B8&useSSL=false"
    String url="jdbc:mysql:"    // 数据库名称
        + "://localhost:3306/db_stu?"    // 端口号+数据库
        + "useUnicode=true&characterEncoding=utf8" // 使用Unicode
        + "&serverTimezone=GMT%2B8&useSSL=false"; // 服务器时区
    Properties info=new Properties();
    info.setProperty("user", "root");    // 数据库用户名
    info.setProperty("password", "123456"); // 数据库密码
    Connection conn=DriverManager.getConnection(url, info); // 连接

    // 3. 显示连接成功与否
    if (conn!=null) {
        System.out.println("数据库连接成功! ");
        // 4. 执行查询
        Statement statm=conn.createStatement();
        String sql="SELECT * FROM tb_student";
        ResultSet rs=statm.executeQuery(sql);
        // 5. 查询到结果, 则打印结果
        while (rs.next()) {
            int id=rs.getInt("id"); // 学号, 结合数据库表字段
            String name=rs.getString("name"); // 姓名

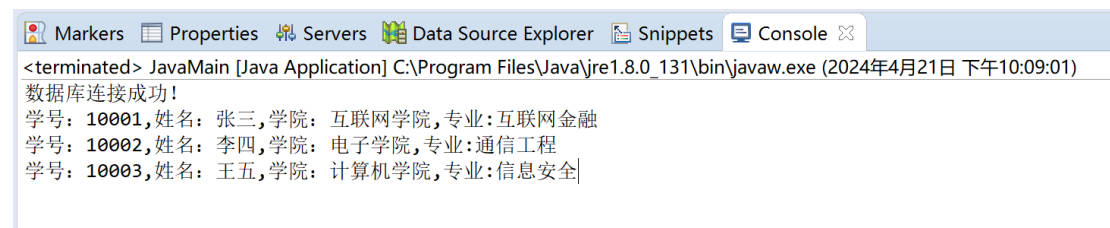
```

```

        String dept=rs.getString("department"); // 学院
        String major=rs.getString("major"); // 专业
        // 控制台打印
        System.out.println("学号: "+id+",姓名: "+name+",学院: "
            +dept+",专业:"+major);
    }
    // 增加一行
    String sql2="INSERT INTO tb_student(id,name,department,major)
VALUES(1005,'李军','管理学院','经济')";
    int n=statm.executeUpdate(sql2);
    if (n>0) {
        System.out.println("数据库成功更新");
    }
    // 删除一行记录
    sql="DELETE FROM tb_student WHERE id =1005";
    n=statm.executeUpdate(sql);
    if (n>0) {
        System.out.println("删除了一条数据记录");
    }
    conn.close();
}
}
}
}
}

```

步骤 4: 程序执行后, 查询结果为:



```

<terminated> JavaMain [Java Application] C:\Program Files\Java\jre1.8.0_131\bin\javaw.exe (2024年4月21日 下午10:09:01)
数据库连接成功!
学号: 10001,姓名: 张三,学院: 互联网学院,专业: 互联网金融
学号: 10002,姓名: 李四,学院: 电子学院,专业: 通信工程
学号: 10003,姓名: 王五,学院: 计算机学院,专业: 信息安全

```